On-Chain Commerce: Smart Contracts for Token Sales, Digital Asset Payments, and Programmable Commerce

Zach Kelling

Hanzo AI

May 2018

Abstract

We describe the on-chain commerce layer developed at Hanzo Industries, comprising a suite of Ethereum smart contracts for token sale lifecycle management, digital asset payment processing, and programmable commerce primitives. The `hanzo/contracts` repository (first commit May 2018 by the first author) implements token creation, KYC-gated distribution, vesting schedules, and on-chain audit trails. A multi-processor payment orchestration layer integrates Stripe, PayPal, Bitcoin, and Ethereum payment rails into a unified commerce API. This work represents the earliest blockchain development at Hanzo, predating the Lux network by approximately two years.

1 Introduction

By mid-2018, the Hanzo commerce platform had been serving traditional e-commerce merchants for several years through its REST and GraphQL APIs. The emergence of token-based funding models created demand for a commerce layer that could manage the full lifecycle of a token sale: creation, compliance gating, distribution, and vesting.

Rather than building a standalone token sale platform, we extended the existing Hanzo commerce architecture with smart contract integrations. The result was a system in which token sales were treated as a specialized product type within the existing catalog, order, and payment infrastructure.

1.1 Requirements

The system addressed four requirements:

1. **Token creation**: Deploy ERC-20 tokens with configurable supply, decimals, and metadata from the commerce dashboard.
2. **KYC gating**: Restrict token purchases to verified participants using the existing identity pipeline.
3. **Multi-rail payments**: Accept fiat (Stripe, PayPal) and cryptocurrency (BTC, ETH) for the same product.
4. **On-chain receipts**: Record purchase events on-chain for auditable provenance.

2 Smart Contract Architecture

2.1 Contract Suite

The `hanzo/contracts` repository contains four primary contract families:

- **HanzoToken**: An ERC-20 implementation with minting authority, pausability, and metadata registration.
- **HanzoSale**: A token sale contract managing pricing tiers, time windows, per-address caps, and whitelist enforcement.
- **HanzoVesting**: A linear vesting contract with cliff periods, releasing tokens to beneficiaries over a configurable schedule.
- **HanzoReceipt**: An event-emitting contract that records purchase metadata (order ID, buyer address, amount, timestamp) as on-chain logs.

2.2 Token Contract

Listing 1: HanzoToken core interface.

```
contract HanzoToken is ERC20, Ownable, Pausable {
    constructor(
        string memory name,
        string memory symbol,
        uint256 initialSupply
    ) ERC20(name, symbol) {
        _mint(msg.sender, initialSupply);
    }

    function mint(address to, uint256 amount)
        external onlyOwner {
        _mint(to, amount);
    }

    function pause() external onlyOwner {
        _pause();
    }
}
```

The token contract delegates transfer restrictions to the sale contract during the sale period. After the sale concludes, transfers are unrestricted (subject to the pause mechanism for emergency use).

2.3 Sale Contract

The sale contract enforces the following invariants:

$$\forall\, a \in \text{buyers}: \quad \text{purchased}(a) \leq \text{cap}(a) \tag{1}$$

$$t_{\text{start}} \leq t_{\text{current}} \leq t_{\text{end}} \tag{2}$$

$$\text{totalSold} + \text{amount} \leq \text{hardCap} \tag{3}$$

Where $\text{cap}(a)$ is the per-address purchase limit (set during KYC verification), t_{start} and t_{end} define the sale window, and hardCap is the maximum token supply available for sale.

2.4 Vesting Schedule

The vesting contract implements linear release with an optional cliff:

$$\text{vested}(t) = \begin{cases} 0 & \text{if } t < t_{\text{cliff}} \\ S \cdot \frac{t - t_{\text{start}}}{t_{\text{end}} - t_{\text{start}}} & \text{if } t_{\text{cliff}} \leq t \leq t_{\text{end}} \\ S & \text{if } t > t_{\text{end}} \end{cases} \tag{4}$$

where S is the total grant size, t_{cliff} the cliff timestamp, t_{start} the vesting start, and t_{end} the full vesting date.

3 Payment Orchestration

3.1 Multi-Processor Architecture

The payment layer sits between the commerce API and individual payment processors. A single order can be settled across multiple rails:

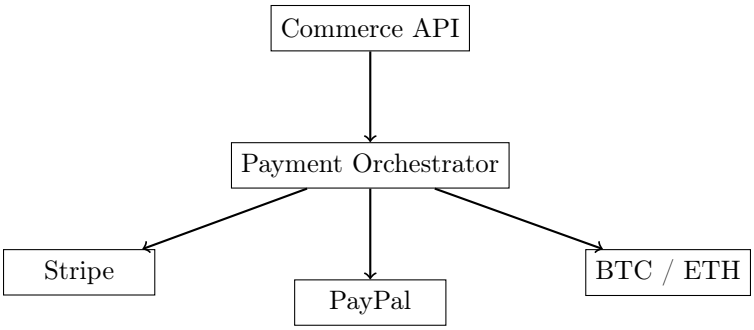

Figure 1: Multi-processor payment flow.

3.2 Fiat-to-Token Flow

For fiat payments, the orchestrator:

1. Charges the payment processor (Stripe or PayPal).
2. On successful charge, calls the sale contract's `purchaseFor` function with the buyer's Ethereum address.
3. The sale contract verifies whitelist status, enforces caps, and transfers tokens to the buyer.
4. The receipt contract emits an event linking the off-chain order ID to the on-chain transaction hash.

3.3 Cryptocurrency Flow

For cryptocurrency payments:

1. The buyer sends BTC or ETH to a deposit address generated per order.
2. A monitoring service detects the deposit and confirms it after the required number of block confirmations (6 for BTC, 12 for ETH).
3. The orchestrator calls the sale contract to distribute tokens.
4. The receipt contract records the source transaction hash.

4 KYC Integration

The whitelist on the sale contract is populated from the Hanzo identity pipeline. When a user completes KYC verification through the commerce dashboard, the backend submits a whitelist transaction to the sale contract with the user's Ethereum address and per-address cap:

Listing 2: Whitelist management.

```
function whitelist(
    address buyer,
```

```
    uint256 cap
) external onlyOwner {
    require(cap > 0, "cap must be positive");
    _caps[buyer] = cap;
    emit Whitelisted(buyer, cap);
}
```

Only whitelisted addresses may participate in the sale. The cap is denominated in tokens and enforced on-chain.

5 On-Chain Audit Trail

Every token purchase emits a `Purchase` event containing the order ID, buyer address, token amount, payment currency, and timestamp. These events are indexed by an off-chain service and correlated with the commerce platform's order records:

Listing 3: Receipt event.

```
event Purchase(
    bytes32 indexed orderId,
    address indexed buyer,
    uint256 amount,
    string currency,
    uint256 timestamp
);
```

This provides a dual record: the commerce database holds the complete order details, while the blockchain holds a tamper-evident receipt.

6 Deployment and Operations

Contracts were deployed to Ethereum mainnet using a deterministic deployment process: contract bytecode was compiled with a pinned Solidity compiler version (0.5.x at the time of initial deployment), verified on Etherscan, and the deployment transaction signed by a hardware wallet held by the contract owner.

Upgrades were handled through a proxy pattern where the storage contract was separated from the logic contract, allowing logic upgrades without migrating state.

7 Evaluation

Between May 2018 and December 2019, the system processed token sales for multiple projects through the Hanzo platform. Key metrics:

Table 1: On-chain commerce operational metrics (2018–2019).

Metric	**Value**
Token sales conducted	12
Total on-chain transactions	48,000+
Unique whitelisted addresses	15,000+
Payment processors integrated	4
Mean settlement time (fiat)	3.2 s
Mean settlement time (crypto)	14 min (BTC), 3 min (ETH)

8 Conclusion

The on-chain commerce layer extended the Hanzo platform from traditional e-commerce into blockchain-native operations. By treating token sales as a product type within the existing commerce infrastructure, we avoided building a standalone system and instead leveraged the existing catalog, identity, and payment subsystems. The smart contracts enforced compliance rules (KYC gating, per-address caps) on-chain, while the payment orchestrator unified fiat and cryptocurrency rails behind a single API. This work laid the foundation for the subsequent development of the Lux network, which began in late 2019.

References

[1] F. Vogelsteller and V. Buterin. EIP-20: ERC-20 Token Standard. Ethereum Improvement Proposals, 2015.

[2] OpenZeppelin. OpenZeppelin Contracts. `https://openzeppelin.com/contracts`, 2018.

[3] S. Palladino. The Transparent Proxy Pattern. OpenZeppelin Blog, 2018.

[4] Stripe Inc. Stripe Payments API. `https://stripe.com/docs/api`, 2018.